

Tester et améliorer une application existante - Partie tests

Dans cet exercice, vous allez analyser et tester le code Java et Angular de l'application EduBibliothèque, à l'aide des outils de test suivants :

Mockito et JUnit pour tester le code back-end en Java

Jest pour les tests unitaires et d'intégration du front-end Angular

Cypress pour les tests end-to-end (E2E) du front-end en Angular

Cet exercice est entièrement guidé.

Vous pouvez suivre les étapes ci-dessous.

Étapes 1 : Analyser le code de test fourni dans le back-end

Lisez attentivement les fichiers de test déjà fournis dans le back-end. Recherchez les mécanismes de test déjà mis en place pour tester certains services et controllers.

Prérequis

Avoir :

relu l'ensemble du code concerné ;

étudié et exécuté les tests de service et de controller pour mieux comprendre leur fonctionnement.

Résultat attendu

Vous avez identifié les tests déjà mis en place.

Recommandations

Lisez le code comme un reviewer.

Les tests déjà disponibles vous serviront d'exemples pour réaliser vos propres tests unitaires et d'intégration.

Étape 2 : Écrire un plan de test

Créez un plan simple de tests à écrire : unitaires, intégration, etc. pour le back et le front.

Prérequis

Avoir :

compris les cas de tests existants ;

identifié les comportements attendus.

Résultat attendu

Une liste de cas de tests à écrire (avec entrées/sorties attendues)

Recommandations

Utilisez la pyramide des tests.

Commencer avec les tests les plus simples.

Points de vigilance

Ne testez pas les cas d'erreur.

Ne vérifiez pas les effets de bord.

Etape 3 : Implémenter les tests unitaires et d'intégration du back-end

Écrivez les tests unitaires et d'intégration pour tester les nouvelles fonctionnalités du back-end.

Vous utiliserez les outils Mockito et JUnit.

Prérequis

Avoir :

fait la liste des tests unitaires et d'intégration à réaliser ;

bien lu les ressources concernant Mockito et JUnit.

Résultat attendu

Tous les services sont testés avec des tests unitaires.

Les nouveaux controllers sont testés avec des tests d'intégration.

Le rapport de couverture de code du back-end est généré : le taux de couverture du back-end a atteint le seuil minimum de 80 %.

Recommandations

Chaque test doit se focaliser sur un cas ou une fonctionnalité précis.

Commencez par les tests unitaires sur les services :

Identifiez les services les plus simples et ajoutez-y les cas de test.

Ensuite, testez les services plus complexes.

Pour chaque service testé, pensez à ajouter assez de tests pour couvrir le plus grand nombre de cas possibles.

Passez aux tests d'intégration sur les controllers :

Des cas de tests d'intégration sont proposés dans la classe `UserControllerTest.java`.

Pour couvrir toute la chaîne de traitement (du controller jusqu'à la base de données), une base de données MySQL est utilisée sous forme de container Docker pour que les cas de test puissent interagir avec une source de données quasi-réelle.

Vous pouvez vous inspirer de cette configuration pour concevoir vos tests d'intégration

Identifiez les controllers les plus simples et commencez à les tester

Ensuite, passez aux controllers plus complexes.

Ajoutez des commentaires dans les tests pour expliquer le test réalisé.

Assurez-vous que le cas de test s'exécute avec succès avant de passer à un autre cas de test.

Etape 4 : Implémentez les tests unitaires et d'intégration du front-end

Ecrivez les tests unitaires et d'intégration pour tester les composants et services du front-end.

Vous utiliserez l'outil Jest.

Prérequis

Avoir lu la documentation sur Jest pour monter en compétences.

Résultat attendu

Tous les services et les composants sont couverts par les tests.

Tous les tests s'exécutent avec succès.

Le rapport montre une couverture de test de 80 % minimum sur la partie front-end.

Recommandations

Commencez toujours par les cas de tests les plus simples.

Réalisez les tests des services.

Étape 5 : Implémentez les tests E2E du front-end
Écrivez les tests E2E pour tester les écrans du front-end.

Vous utiliserez l'outil Cypress.

Prérequis

Avoir lu la documentation sur Cypress.

Résultat attendu

Tous les écrans sont couverts par les tests E2E.

Tous les tests E2E s'exécutent avec succès.

Le rapport montre une couverture de test de 80 % minimum sur la partie E2E.

Recommandations

Commencez les tests E2E par les formulaires les plus simples tels que la création de compte et la connexion.

Évoluez ensuite vers les pages plus complexes.

Mockez les appels d'API pour faire fonctionner les tests E2E

Points de vigilance

Avant de passer à l'implémentation d'un nouveau test E2E, assurez-vous que le précédent fonctionne normalement.